Tema 4:

Metode de dezvoltare ale SI. Metoda OO. Limbajul UML

Natalia Pleșca,

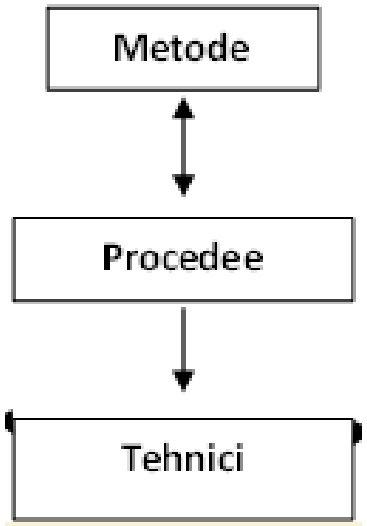
Lector universitar

Conținut

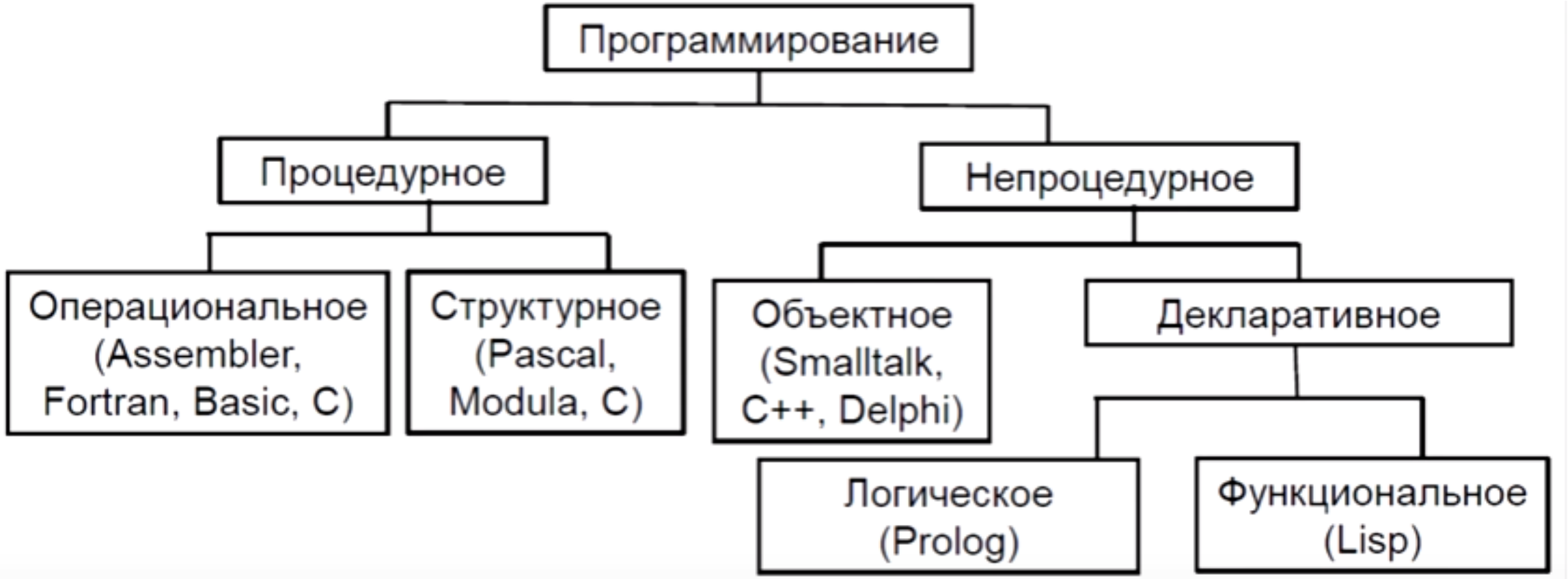
- Metode de dezvoltare ale SI
- Principii utilizate în cadrul fiecărei metode
- Metoda OO
- Limbajul UML

METODE DE DEZVOLTARE

- În procesul dezvoltării SI se utilizează: metode, tehnici, instrumente, procedee de lucru
- **Metodele de dezvoltare** ale SI reprezintă modul în care analiştii de sistem, programatorii şi alte persoane implicate, realizează procesul de analiză, proiectare realizare şi implementare a SI



Paradigme de programare



METODA STRUCTURATĂ

Metoda structurată (procedurală) este considerată **funcțional-orientată**

- Sistemul informațional şi cel informatic se structurează în baza funcțiilor sale
- Are la bază analiza funcțională - fiecare funcție identificată se divide ierarhic în subfuncții, continuând în această manieră până se ajunge la componente suficient de mici, astfel încât acestea să poată fi programate cu uşurință
- Se utilizează metoda Jackson – structura unui sistem soft derivă din structura datelor «Алгоритмы + структуры данных = программы» Wirth, 1976

Metoda structurată conduce la construirea a două tipuri de modele la etapa de analiză şi cea de proiectare:

- **Modelul funcțional:** SADT (Structured Analysis and Design Technique), DFD (Data Flow Diagram), JSD (Jackson System Development),
- **Modelul datelor**: ERD (Entity-Relationship Diagrams)

METODA STRUCTURATĂ

- Prin analiză şi proiectare structurată se înțelege o metodă de cercetare a sistemului, care începe cu o descriere generală şi continuă cu o detaliere treptată a lui în descrierea aspectelor de comportament şi funcționare. Modelul general al sistemului este prezentat ca o structură ierarhică, ce reprezintă diferite nivele de abstractizare, cu un număr limitat de componente la fiecare nivel
- Esența metodei structurate constă în descompunerea SI în funcții care pot fi automatizate – sistemul este împărțit în subsisteme funcționale, care la rândul său se divid în subfuncții/sau probleme. Procesul descompunerii este finalizat atunci când se ajunge la proceduri concrete, care uşor pot fi transformate în coduri de program

METODA OO

Metoda se consideră orientată **spre date**:

- se acordă o atenție deosebită structurii datelor (în ultimul timp şi funcțiilor)
- sistemul informațional şi cel informatic este perceput ca o structură de obiecte autonome, ce se organizează şi cooperează între ele
- un obiect are un anumit comportament, definit prin ansamblul operațiilor (serviciilor) pe care le poate efectua
  - datele şi prelucrările prin care este implementat acest comportament sunt încapsulate (mascate) şi sunt inaccesibile celorlalte obiecte
- fiecare obiect poate participa la compunerea altor obiecte mai complexe
- fiecare obiect poate interveni în mai multe funcții sau scenarii funcționale diferite

Modele în cadrul metodei OO

Metoda OO conduce la construirea a 3 tipuri de modele:

- ***Modelul static*** (*structural*) are rolul de a descrie obiectele care intervin în problema de rezolvat şi relațiile dintre ele. Modelul reprezintă descrierea structurii statice a obiectelor, claselor de obiecte, a operațiilor şi atributelor, precum şi ale relațiilor dintre ele
- ***Modelul dinamic*** are rolul de a descrie
  - stările pe care le poate avea un obiect şi evenimentele, ce se produc la trecerea dintr-o structură în alta
  - interacțiunea dintre obiecte (prin schimb de mesaje) şi este focalizat pe aspecte ce se schimbă în timp
- ***Modelul functional / comportamental*** are rolul de a descrie prelucrările şi fluxurile de date. Modelul funcțional prezintă transformările valorilor datelor, precizând sursa şi destinația lor

IDEEA MODELĂRII ÎN METODA OO

- În proiectarea OO accentul se pune pe *modelarea* obiectelor şi a relaţiilor dintre ele
- Sistemul modelat se descompune în obiecte cât mai simple, care se modeleaza separat
- În procesul de modelare a obiectului se ajunge la date și algoritmi (similar metodei structurate), deoarece starea obiectului este reprezentată prin *date*, iar comportamentul acestuia - prin *metode*, care sunt ele însele *subprograme*, deci conțin - de asemenea - date locale și algoritmi
- Apoi, toate aceste modele se combină, pentru a reprezenta sistemul în ansamblu

Principiile de bază utilizate în metoda OO

- **Abstractizare** (*Abstraction*): Reprezintă procesul de simplificare a complexității prin evidențierea doar a caracteristicilor esențiale ale unui obiect și ascunderea detaliilor irelevante
  - Exemplu: Clasa ***Masina*** definește doar proprietăți generale (marca, viteza), fără a detalia cum funcționează motorul
- **Încapsulare** (*Encapsulation*): Se referă la ascunderea datelor interne ale obiectului și accesul la ele doar prin metode publice (*get/set*). Se face cu scopul de a proteja integritatea datelor și reducerea dependențelor între componente
- **Moștenire** (*Inheritance*): Permite unei clase să preia proprietăți și metode de la o altă clasă (*superclasă*). Avantajul utilizării acestui principiu este reutilizarea codului și extinderea funcționalităților fără duplicare
- **Polimorfism** (*Polymorphism*): Oferă posibilitatea de a utiliza aceeași interfață pentru obiecte diferite, care se comportă în mod specific
  - Exemplu: metoda *deseneaza()* poate fi definită diferit în clasele *Cerc*, *Patrat* sau Triunghi

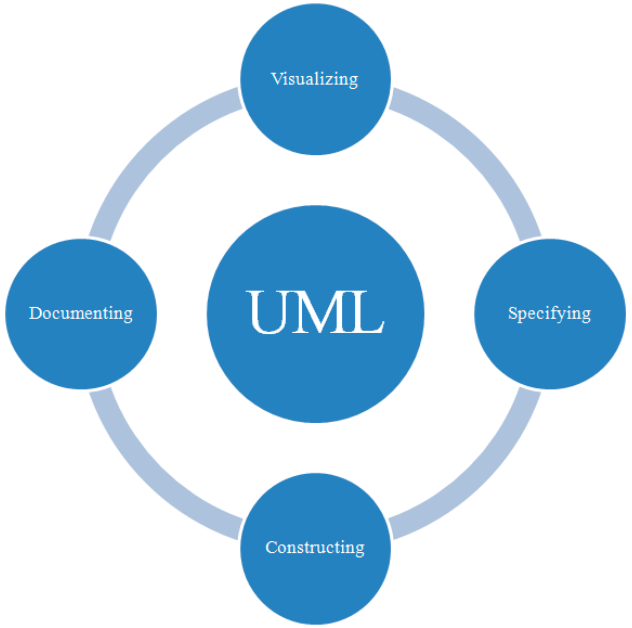
NOȚIUNI SPECIFICE în OO

- **Clasificarea** presupune că obiectele care au structuri de date (atribute) şi comportament (operații) identice, se grupează în clase
- O **clasă** este o abstractizare, care descrie proprietățile şi comportamentul unui tip de obiect
- Fiece clasă descrie o mulțime de obiecte individuale, numărul cărora poate fi şi infinit
- Toate obiectele din această mulțime sunt instanțe sau exemplare ale clasei. Procesul creării unui obiect este numit **instanțiere**
- În funcție de nivelul de abstractizare, clasele pot forma o structura ierarhică
  - Cu cât o clasa se afla în aceasta ierarhie la un nivel mai înalt, cu atât ea este mai abstractă, deci descrie un număr mai mic de atribute ale obiectelor care îi aparțin

Exemplu de grupare a obiectelor în clase de obiecte

- **Membru** (idMembru, nume, prenume, email, telefon, adresa, stareMembru {Activ, Suspendat}, tipAbonament, soldRestant (Bani), poateImprumuta(): bool
- **CardMembru** (idCard, codBare, dataExpirare, stareCard {Activ, Blocată},
- **Bibliotecar** (idBibliotecar, nume, roluri {Birou, Administrator}
- **Imprumut** (idImprumut, dataStart, dataScadenta, dataReturnare, penalizareZilnica (Bani):prelungeste(); inchide(dataReturnare); calculeazaAmenda(dataCurenta)
- ....

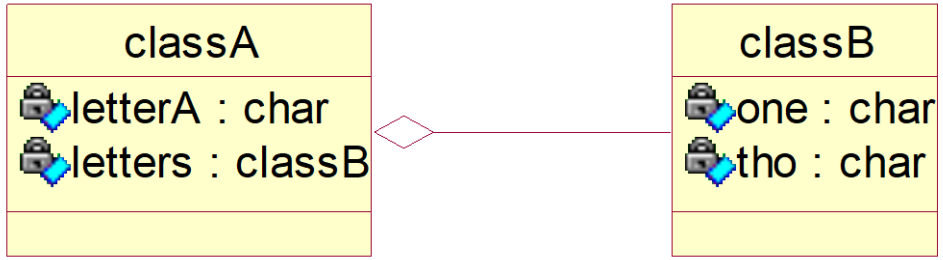
Limbajul UML



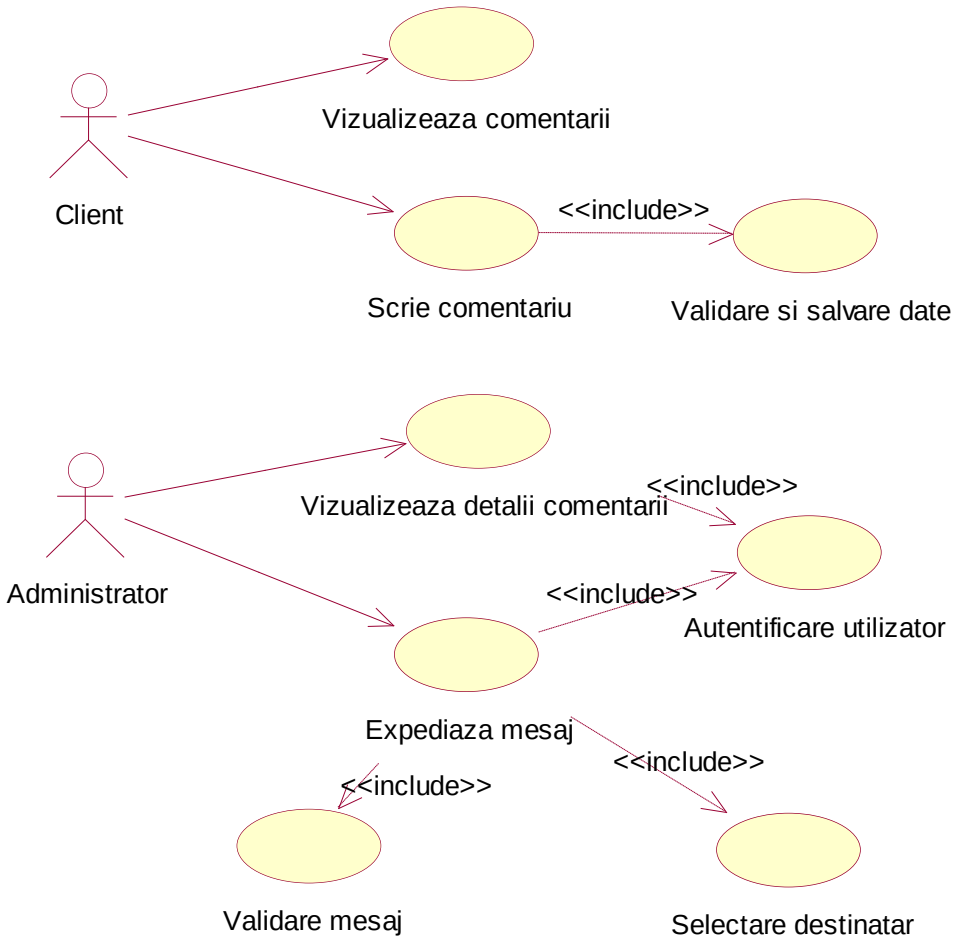
- UML (*Unified Modeling Language*) - este un acronim care înseamnă **limbajul de modelare unificat**
- UML este o abordare modernă pentru modelarea și documentarea software-ului, reprezentând una dintre cele mai populare tehnici de modelare a proceselor de business
- Limbajul UML utilizează pentru modelare diagramele, care au scopul de a reprezenta vizual un sistem, împreună cu principalii actori, roluri ale acestora, acțiuni, artefacte sau clase, pentru a înțelege mai bine, a modifica, întreține sau documenta informațiile despre sistem
- Cum spune vechiul proverb: „o imagine valorează cât o mie de cuvinte”
- Utilizând reprezentări vizuale, membrii echipei de dezvoltare sunt capabili să înțeleagă mai bine posibilele neajunsuri sau erori în procesele software sau în cele de business

Exemplu. Analogie cod și diagrama claselor

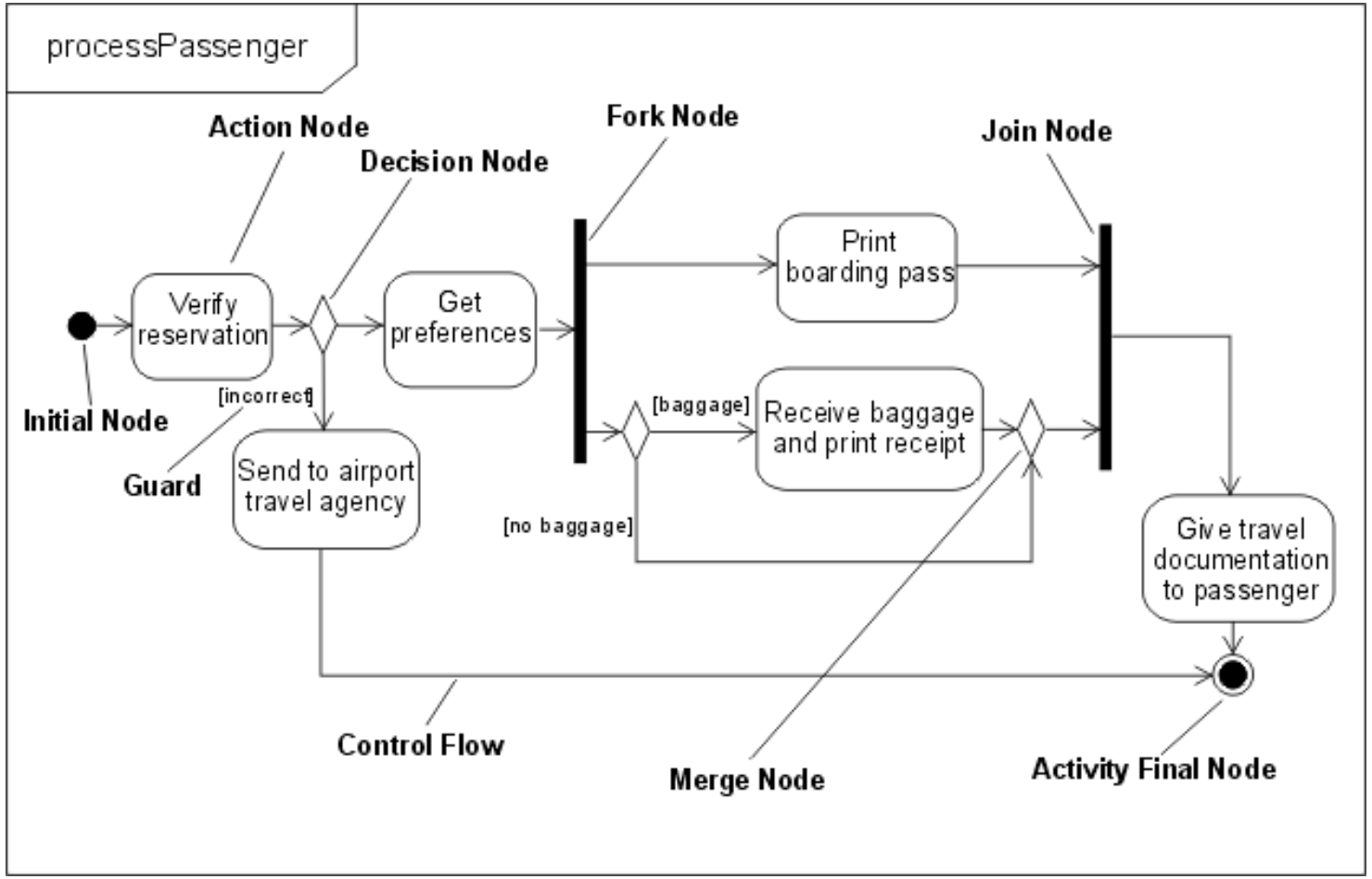
```
public class ClassA
{
    char letterA;
    ClassB letters;
}

public class ClassB
{
    char one;
    char two;
}
```



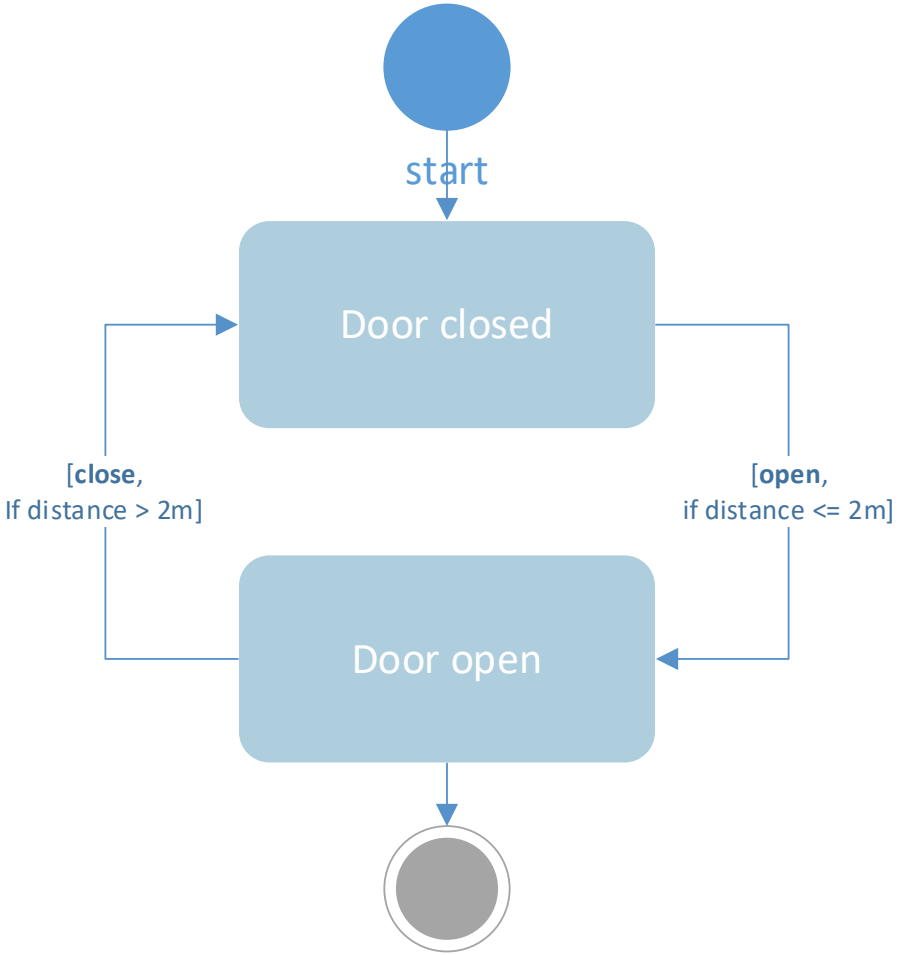
Sau un alt exemplu – prezentare grafică a funcționalităților



Prezentarea acțunilor unei activități



Un exemplu de diagrama de stari



Istoric

- UML a fost creat ca urmare a haosului care era în jurul proceselor de dezvoltare și documentare a produselor software. În anii 90, au existat mai multe moduri de a reprezenta și documenta sistemele software
- UML a fost dezvoltat de trei ingineri software ("Three Amigos" - Grady Booch (creator of Booch method), Ivar Jacobson (Object-Oriented Software Engineering, OOSE), and Jim Rumbaugh (Object-Modeling Technique, OMT)), care lucrau la Rational Software. Ulterior, a fost adoptat ca standard în 1997, având câteva actualizări de atunci
- Limbajul UML poate fi utilizat pentru:
  - Proiectarea arhitecturii fizice și logice a sistemului software sau informatic
  - Proiectarea detaliată a componentelor sistemului - multe probleme de design sau defecte, ale softului pot fi descoperite analizând modelele și corectându-le
  - Proiectare inversă - după scrierea codului, pot fi generate diagramele UML, pentru a documenta diferite activități ale actorilor și fluxuri de lucru

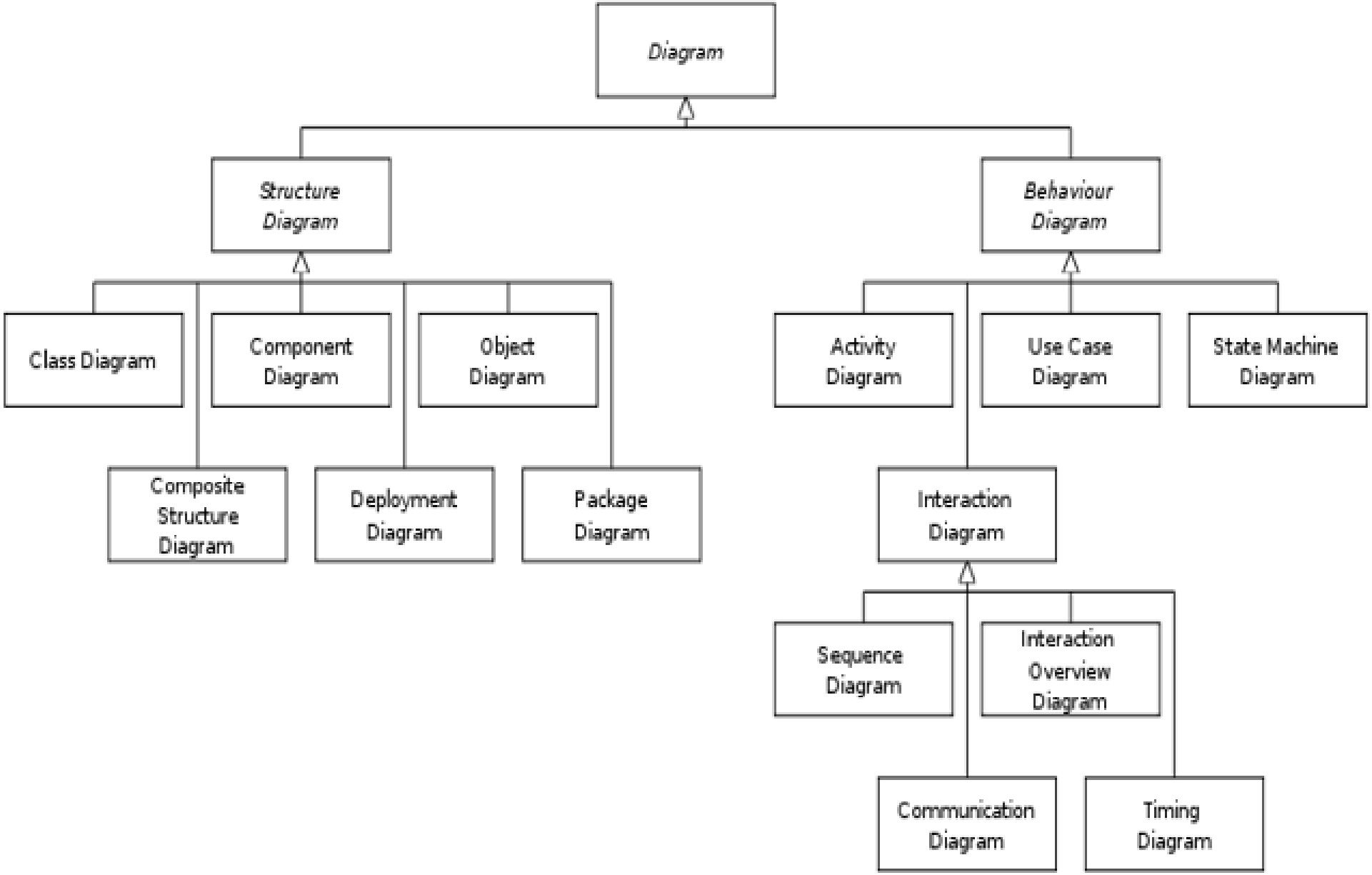
Utilizare

- UML – nu este un limbaj de programare, ci unul de modelare!!!
- UML poate servi pentru a documenta soluții de programare pentru limbajele de programare OO, precum: Java, C ++, C#, PHPOO, Python etc.
- Există și instrumente CASE (Rational Rose), care pot transforma diagrame UML în carcase de cod de program
- Chiar dacă UML a fost creat pentru modelarea sistemelor software, acesta a găsit mai multe utilizări în domeniul businessului sau a sistemelor non-software

Tipuri de diagrame UML

- UML permite reprezentarea sistemului folosind mai multe tipuri de **diagrame**, care se completează una pe cealaltă
- Combinând diverse tipuri de diagrame UML se elaborează o vedere/ model completă asupra staticii sistemului (structurii acestuia) şi asupra dinamicii (comportamentului) sistemului
- După cum sugerează și denumirea, unele diagrame UML încearcă să analizeze și să descrie structura sistemului sau a procesului, în timp ce celelalte - descriu comportamentul sistemului, actorii săi etc.

Clasificarea diagramelor UML



DIAGRAME COMPORTAMENTALE

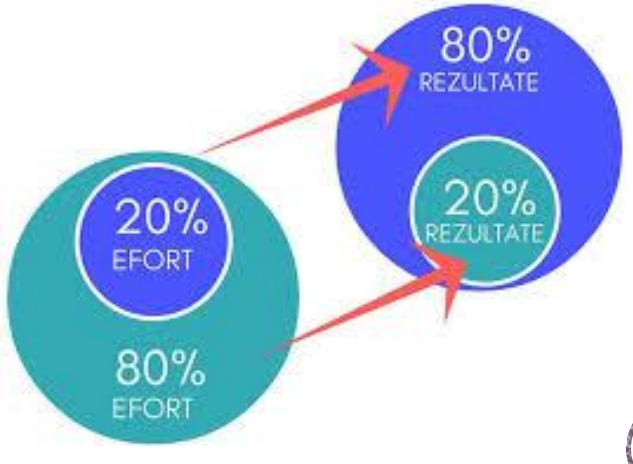
- Seven Diagrams:
  - Activity **Unchanged**
  - Use Case **Unchanged**
  - State Machine **Minor Renaming**
  - Interaction
    - Timing **New**
    - Sequence **Unchanged**
    - Communication **Minor Renaming**
    - Interaction Overview **New**

DIAGRAME DE STRUCTURĂ

- Six Diagrams:
  - Package Diagram **Unchanged**
  - Object Diagram **Unchanged**
  - Composite Structure Diagram **New**
  - Component Diagram **Unchanged**
  - Deployment Diagram **Unchanged**
  - Class Diagram **Unchanged**

Cele mai utilizate diagrame UML

- Nu toate cele 13 tipuri diferite de diagrame UML sunt utilizate în mod regulat la documentarea sistemelor și / sau arhitecturilor de sistem
- **Principiul Pareto** pare să se aplice și în ceea ce privește utilizarea diagramelor UML - 20% din diagrame sunt folosite de 80% din dezvoltatori ☺
- Cele mai frecvent utilizate diagrame UML în dezvoltarea de software sunt:
  - diagramele cazurilor de utilizare,
  - diagramele claselor
  - diagramele activităților
  - diagramele de secvență



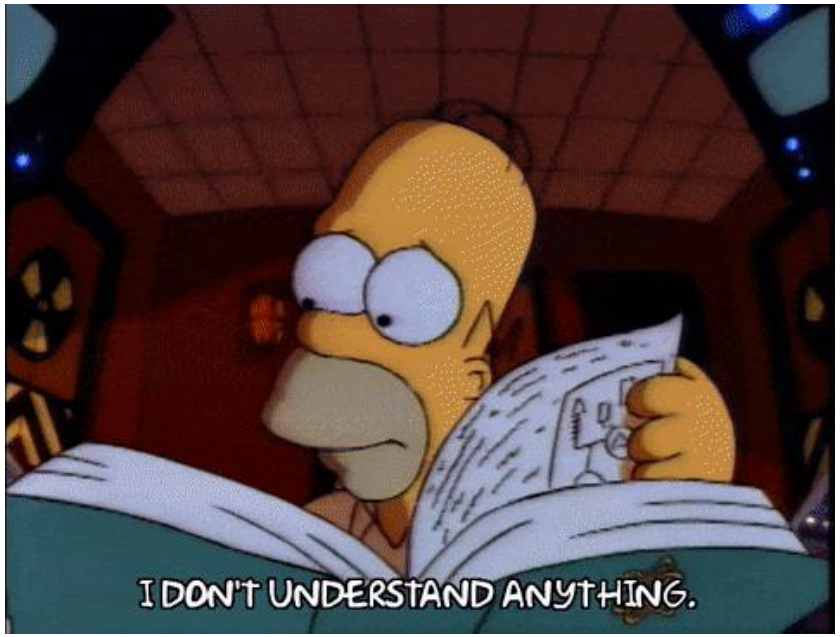
Hint

- **Principiul Pareto** (cunoscut de asemenea ca **regula 80/20** sau **legea celor puțini, dar critici**) prevede că, pentru multe evenimente, aproximativ 80% din efecte sunt produse de 20% din cauze
  - Pareto arăta că aproximativ 80% din suprafața Italiei, era deținută de 20% din populație. Pareto a dezvoltat principiul, observând că aproximativ 20% din păstăile de mazăre în grădina lui, produceau 80% din recoltă
- Este o regulă generală, frecvent utilizată în domeniul afacerilor; de exemplu, „80% din vânzări provin de la 20% din clienți”
- Din punct de vedere matematic, regula 80/20 este aproximativ urmată de distribuția Pareto, pentru un anumit set de parametri, și s-a demonstrat empiric că multe fenomene naturale prezintă o astfel de distribuție ☺
- Cei mai bogați 20% din populație, dețin 82,70% din venituri (1989) - statistica

Editoare pentru construirea diagramelor

- online: https://online.visual-paradigm.com/diagrams/
- MS Visio
- Rational Rose – CASE
- https://sparxsystems.com/products/ea/downloads.html
- https://app.diagrams.net/

!!!

- **3** idei/noțiuni importante învățate azi
- **2** întrebări/neclarități care au apărut
- **1** sugestie pentru tema următoare

Data viitoare scriem testul 1